prj1 → prj2 代码阅读提纲

你已经掌握期中提纲里的特权级、PCB、调度与同步。接下来要能沿着一条真实执行路径，说明**谁调用谁、此刻在哪个栈、哪些状态被保存、下一条指令从哪里开始**。

本文将实验一称为prj1、实验二A-Core称为prj2。源码依据为 `zxymao/Chatgpt` 的 `projects/Project2/`，版本为 `4d6cb77cd2ca`。prj2来自已校验的源码包；prj1依据仓库保存的实验一对照差异恢复，反向恢复后再正向应用，已核对与源码包一致。这里只规范化了文本换行。仓库里的 `os-lab/` 是另一套教学内核。

先抓住两种执行模型。 prj1是“内核接收任务名 → 读入镜像 → 调用入口 → 任务返回 → 等待下个名字”。prj2是“预先加载多个任务 → 为每个任务建立PCB和栈 → 按事件保存当前执行流 → 恢复另一个执行流”。

建议按这六轮阅读

1. 第2—4页读prj1：搞清源码、ELF、磁盘镜像、内存中的任务四者的关系。
2. 第5—7页读prj2初始化：只跟第一个任务，画出它从PCB变成实际执行流的过程。
3. 第8页跟主动yield：手算A、B两次切换，特别注意各自的内核栈。
4. 第9页跟锁：让A持锁时yield，观察B为什么必须进入阻塞队列。
5. 第10—11页跟ecall：把用户库、异常入口、系统调用表、返回用户态串成一条线。
6. 第12—14页跟睡眠和时钟，再用自测题核对自己能否独立解释代码。

每次只读一个小节所列的文件。先写出函数的输入、共享状态和返回位置，再看函数体。能用两三个任务画出一次完整交错，比把所有头文件逐个读完更有用。

阅读时始终分清的五组对象

- `tasks[]` 是镜像任务元信息；`pcb[]` 才是运行时调度状态。
- `entrypoint` 是程序首次入口；`ra` 是函数返回地址；`sepc` 是异常返回位置。
- `switchto_context_t` 保存一次函数边界上的切换现场；`regs_context_t` 保存异常现场。
- `ready_queue` 决定哪些任务可运行；锁等待队列和 `sleep_queue` 存放暂时不能运行的任务。
- 库函数、内核函数、BIOS接口位于不同层；名字里出现 `sys_` 并不能单独证明发生了特权级切换。

配套的 **源码索引.html** 收录P1、F2、P2源码文本，可搜索文件名、函数名或TODO。正文中的“版本 文件:行号”链接直接定位到对应源码行；P1表示实验一，F2表示原框架，P2表示完成版。

重点补代码： 第15页起增加TODO导读，第16页起的A1—A20按原TODO配上重要实现、输入输出、不变量和常见错误。F2表示原始实验二框架；完整TODO清单也可逐条定位。建议读完一个机制，就立即对照相应代码组。

完成目标： 合上代码后，能画出“加载一个任务”“第一次运行一个任务”“一次yield”“一次锁阻塞”“一次时钟抢占”五条路径。

1. prj1：先看构建出来的东西是什么

阅读入口：P1 Makefile:1、P1 tools/createimage.c:87、P2 tools/createimage.c:51。

`Makefile` 先用RISC-V交叉编译器生成bootblock、内核和任务的ELF；镜像工具 `createimage` 则是运行在宿主Linux上的普通程序，用宿主GCC编译。二者的执行机器不同。

镜像工具读取ELF头和Program Header，把 `PT_LOAD` 中的文件数据写入镜像；ELF调试信息和文件头不是任务执行时要逐字加载的代码。`p_filesz` 表示文件中已有的数据，`p_memsz` 包含运行时还需要的内存；BSS的清零由启动代码完成。



读取位置由字节偏移描述；按512B扇区读取后，还要处理扇区内偏移

磁盘镜像的顺序布局

bootblock只占第0扇区，内核从镜像字节偏移512开始。后续任务按实际文件跨度连续写入，因此任务起点不保证落在扇区边界上。prj2额外将镜像尾部补到扇区边界。

引导区尾部的四组信息是镜像工具与启动代码之间的共同协议：`0x1f6`存2字节任务数，`0x1f8`存4字节任务表字节偏移，`0x1fc`存2字节内核字节数，`0x1fe`存 `55 aa` 签名。bootblock装入 `0x50200000` 后，启动代码就在该基址加上述偏移的位置读取它们。

P1 include/os/task.h:16 的 `task_info_t` 包含32字节名称、4字节offset、4字节size、8字节entrypoint，共48字节。**offset指磁盘镜像位置，entrypoint指执行地址，二者不能互换。**

prj1的 `write_segment()` 把各加载段的文件字节顺序写入。prj2的 `append_elf()` 则按 `p_vaddr-expected` 放置各段并填充中间空洞，保留相对地址布局。镜像工具的这项变化与调度算法无关，但会影响加载后的代码是否位于链接器预期地址。

本工程常见地址：bootblock为 `0x50200000`，内核入口为 `0x50201000`，任务从 `0x52000000` 起按 `0x10000` 分槽。任务放在不同槽位，并不自动等于有页表隔离。

读完应能回答： 为什么不能直接把完整ELF文件当机器指令运行？任务表为什么还要保存size和entrypoint？内核大小字段存的是字节数还是扇区数？

2. prj1：从bootblock走进C函数

阅读顺序：P1 arch/riscv/boot/bootblock.S:17 → P1 arch/riscv/kernel/head.S:10 → P1 init/main.c:50。

```
课程固件 / U-Boot
  → bootblock 的 main
  → BIOS读磁盘，把内核装到0x50201000
  → head.S 的 _start：清BSS、建立sp
  → 内核 init/main.c 的 main(tasknum, app_info_offset)
```

bootblock先通过固定BIOS入口 `0x50150000` 打印和读盘。读内核时，a0是目标地址，a1是扇区数，a2是起始扇区1，a7是BIOS服务号。扇区数由 `(kernel_bytes+511)>>9` 向上取整。调用BIOS后，再把任务数放入a0、任务表偏移放入a1，最后 `jr` 跳到内核入口。

因此，同一个a0在不同调用点有不同含义。读代码要沿当前调用约定追踪，不能把某个寄存器永久理解成“任务号寄存器”。

这部分汇编先认清：`li`装入常数，`la`取符号地址，`ld/sd`读写64位内存，`call`写ra后跳转，`ret`按ra返回，`csrw`写CSR。`ld sp,...` 改变的是栈指针，不能理解成“把另一个任务的整块栈复制过来”。

`head.S` 处理C函数能成立的前提：关闭相关中断使能、用链接器符号定位BSS并清零、把sp设为启动栈，然后 `call main`。清BSS时使用t0/t1，保留了bootblock传入的a0/a1。prj1的 `bss_check()` 随后检查全局数组是否为零，是对启动工作的直接观察。

三个入口名相似，但职责不同

- `bootblock.S: main` 是引导块入口，负责装入内核。
- `kernel/head.S: _start` 是内核运行环境入口，随后调用内核C语言main。
- `crt0/crt0.S: _start` 是任务入口，清任务自身的BSS，再调用任务C语言main。

任务入口见 P1 arch/riscv/crt0/crt0.S:4。prj1的crt0保存ra和s0，清BSS后调用任务main；任务main返回时恢复ra/s0并返回加载器。它沿用当前执行流的栈，没有在这里创建可独立调度的任务栈。

关键区别： `call/jalr` 为通常的函数返回准备ra；`jr`只是跳转。内核给任务入口发起函数调用后，crt0必须维护这条返回路径，任务结束才会回到交互循环。这里还没有“任务结束后由调度器选择另一个PCB”的机制。

动手画图： 给bootblock、内核启动、任务crt0各画一个框，在跨框箭头旁标明“普通跳转 / 函数调用”，再标出a0/a1的含义。不要把课程BIOS服务号与后面prj2的系统调用号混为一张表。

3. prj1：一个任务名如何变成正在运行的指令

核心入口：P1 init/main.c:106、P1 kernel/loader/loader.c:12、P1 kernel/loader/loader.c:35。

交互循环从BIOS读取字符，把输入保存在 `taskname[]`；遇到回车补零，空输入重新等待。`load_task_img(taskname)` 遍历 `tasks[]` 比较名称，匹配后把指定镜像字节读到 `entrypoint`，然后返回入口地址。找不到时返回0，main打印错误后继续等待。

以下是实际执行点的源码节选：

```
129         uint64_t entry=load_task_img(taskname);
130         if(entry==0){
131             bios_putstr("Task not found\n\r");
132             continue;
133         }
134         ((void (*)(void))entry)();
135         bios_putstr("\n\rTask finished, back to loader\n\r");
```

这里把整数入口地址转换成函数指针并调用。**只有一个执行流正在沿着这条调用链运行**：main的当前调用栈还在，任务返回后继续执行“Task finished”。存在多个任务镜像并不表示已经有多个可恢复的运行上下文。

跨扇区读取要跟一次具体数字

`load_img_bytes(offset,size,dst)` 每轮计算：所在扇区为 `offset/512`，扇区内偏移为 `offset%512`，本轮复制长度为 `min(512−偏移,size)`。先把整扇区读进临时buffer，再复制需要的字节，最后同时推进offset和目标指针，减少剩余size。

例如从镜像字节偏移1000读取600字节：第1轮从扇区1的偏移488处复制24字节；第2轮从扇区2复制512字节；第3轮从扇区3复制64字节。最终恰好读取600字节。这里的24、512、64是阅读示例，不是程序固定的任务大小。

`tasks[]` 本身也通过这个函数读入，因此任务表同样可以从非扇区边界开始。prj2仍保留此思路，并在任务代码加载完成后执行 `fence.i`，使随后取指能看到刚写入的代码。

把printf下面的层次先分清

prj1的 `bios_putstr → call_jmptab` 根据固定跳转表中的函数指针调用内核包装函数，包装函数再进入课程BIOS。参见 P1 tiny_libc/include/kernel.h:12 和 P1 arch/riscv/bios/common.c:7。这是函数指针间接调用；不能把它直接当成“用户态ecall陷入内核”。

读完应能独立复述： 谁填写 `tasks[]`？从名称找到入口需要哪四项信息？任务为何能返回加载器？把offset错误地当成内存地址会在哪一层出错？

4. prj2：从单任务调用改成多任务启动

先读 P2 Makefile:14、P2 init/main.c:188，再按下面顺序展开函数体。

```
main
 → init_jmptab → init_task_info → init_pcb
 → 读取time_base → init_locks → init_syscall
 → init_exception → init_screen
 → 阶段4设置timer、使能STIE → idle循环中的do_scheduler
```

prj2启动时把选定的任务都加载好并加入就绪队列，不再等待用户逐个输入任务名。`init_task_info()` 检查任务数、名称终止符、入口槽位和大小；`init_pcb()` 才进一步为任务分配运行时资源。

阶段开关会改变执行路径，跟代码前先看编译时的 `LAB_STAGE`：

阶段	任务方式	主要切换来源	首先跟读的测试
1	S模式，服务经跳转表	主动yield	`coop1/coop2`
2	S模式，服务经跳转表	yield及锁阻塞	`mutex1/mutex2`
3	U模式，服务经ecall	yield、阻塞和睡眠	`sleep/coop_timer`
4	U模式，服务经ecall	加上时钟抢占	`probe1/probe2`

上表是本仓库A-Core版本的实现安排。Task1/2、完整Task3、Task4在同一份源码中用条件编译区分；预处理后的机器代码只包含当前阶段的分支。默认阶段是4。

`init_pcb()` 的单任务初始化顺序为：清PCB → 分配pid → 标READY → 初始化链表节点 → 分配一页内核栈及一页用户栈 → 加载任务 → 构造初始栈 → 进入ready_queue。见 P2 init/main.c:146。

内存分配器见 P2 kernel/mm/mm.c:6：它只是将当前位置按4096字节对齐并向前推进。这里的“用户页/内核页”是两段分配区域，不代表已经建立不同页表；本工程没有完整的地址空间隔离、回收式页分配器或SMP调度。

这一页只需形成一个判断： prj1读入任务后立刻调用入口；prj2读入任务后先放进“能在未来被恢复”的状态，真正开始执行要等调度和汇编恢复路径。

5. PCB、链表与三个队列不变量

阅读入口：P2 include/os/sched.h:38、P2 include/os/sched.h:65、P2 include/os/list.h:43。

`tasks[i]` 描述磁盘里有什么；`pcb[i]` 描述现在执行到什么状态。PCB中的核心字段应按“被谁使用”理解：

- `kernel_sp`：汇编恢复该任务内核执行现场时要取出的栈位置。
- `user_sp`：U态陷入时记录用户栈指针；返回时仍须恢复原用户sp。
- `trap_t0`：异常入口最早借用t0之前的临时保存位置。
- `user_stack_base`：栈区域边界，也用于本版本的输出指针检查。
- `list`与`status`：队列操作和调度选择共同维护运行资格。
- `pid`、`cursor_x/y`、`wakeup_time`：身份、屏幕位置和睡眠截止tick。

`current_running` 不是普通全局指针，它绑定到RISC-V的tp寄存器。S模式的内核按该约定使用tp；阶段3/4进入U态后，用户自己的tp可以不同。因此异常入口还需要 `sscratch` 帮助重新找到PCB，不能在任意特权级都把tp当成PCB。

链表不是“PCB指针数组”

每个PCB内嵌一个 `list_node_t`。空链表的头节点指向自己；脱链的PCB节点也初始化为自环。`list_entry(node,pcb_t,list)` 减去成员偏移，从节点地址还原整个PCB。`list_del()` 删除后重新初始化节点，便于后续判断它是否已经脱链。

跟读 P2 include/os/list.h:53 的五个小函数：`list_init`、`list_empty`、`list_add_tail`、`list_del`、`list_pop_front`。先手画一个头节点和A、B两个PCB节点，再执行一次“弹出队头，追加队尾”。

读调度代码时不断核对

1. 运行中的普通任务已从ready_queue取出，不应同时还在该队列中。
2. 阻塞任务的同一个list节点只挂在某一个等待队列中；它不能同时在就绪队列和锁等待队列里。
3. `do_unblock()` 只把BLOCKED改成READY并加入队尾，不承诺马上执行该任务。

pid0有单独PCB，负责启动及idle，不与普通任务一样每次回到就绪队列。`TASK_EXITED` 虽然存在于枚举中，本版本没有实现完整的进程退出系统调用。

自查： 如果只改status、不移动链表，会发生什么？如果只把B从锁队列摘掉、却没加入ready_queue，B还能被调度到吗？

6. 新任务第一次执行：先构造一个可以恢复的栈

阅读入口：P2 init/main.c:124、P2 arch/riscv/include/asm/regs.h:85、P2 arch/riscv/kernel/entry.S:167。

新任务从未进入过异常处理，也从未调用过 `switch_to()`。初始化代码就预先构造两种现场，让后续通用恢复代码能够把它启动起来。



新任务初始内核栈

288字节的 `regs_context_t` 包含32个通用寄存器槽位和4个CSR槽位；112字节的 `switchto_context_t` 包含ra、sp、s0—s11。两者都位于该任务的内核栈中，但代表不同层次的恢复状态。

`regs->regs[2]=utop` 设置恢复后的任务sp，`regs->sepc=entry` 设置首次执行位置。阶段3/4设置 `sstatus=SR_SPIE`，SPP为0，因此sret进入U模式；阶段1/2设置SPP，恢复到S模式，并在保存的tp槽位放入PCB地址。

下面四行把切换现场接到异常返回现场上：

```
140      sw->regs[0] = (ptr_t)ret_from_exception;
141      sw->regs[1] = (ptr_t)sw;
142      task->kernel_sp = (ptr_t)sw;
143      task->user_sp = utop;
```

第一次启动任务的路径因此是：

```
do_scheduler选中该PCB
 → switch_to加载它的kernel_sp，恢复初始切换帧
 → ret跳到ret_from_exception
 → RESTORE_CONTEXT恢复预制异常帧
 → sret使用sepc/sstatus进入任务crt0
 → crt0清BSS，再call用户main
```

首次启动的位置由sepc控制，不是靠直接call任务入口。 参见 P2 arch/riscv/crt0/crt0.S:3：prj2的任务main如果意外返回，crt0会反复 `sys_yield()`，因为这里不存在prj1那样真实的加载器调用者。

图只表示新任务的初始布局。任务在内核调用链中再次被切走时，`kernel_sp` 会指向当前更深位置的切换帧；不能把它永久理解成固定栈顶或固定288字节帧地址。结构偏移的 `_Static_assert` 正是为了让C布局与汇编约定一致。

7. 一次yield：选择下一个任务，再真的换栈

先读 P2 kernel/sched/sched.c:16，再读 P2 arch/riscv/kernel/entry.S:130。

`do_scheduler()` 保存中断状态并关本地中断，先检查到期睡眠者。当前普通任务仍为RUNNING时，将它改为READY并放到队尾；已BLOCKED的任务不回就绪队列。随后弹出队首作为next，队列为空则选择pid0；只有 `prev!=next` 才调用汇编切换。

假设A正在运行、队列中只有B：A yield时先把自己排到B之后，弹出B，于是B运行而队列只剩A。B再次yield后，A又被选中。这是FIFO轮转，不是每次扫描所有PCB并凭pid大小选择。

`switch_to(prev,next)` 中最关键的片段如下，中间省略部分同类寄存器读写：

```
130  ENTRY(switch_to)
131      addi sp, sp, -SWITCH_TO_SIZE
132      sd ra, 0(sp)
133      sd sp, 8(sp)
134      sd s0, 16(sp)
135      sd s1, 24(sp)
    …
146      sd sp, PCB_KERNEL_SP(a0)
147      mv tp, a1
148      csrw CSR_SSCRATCH, tp
149      ld sp, PCB_KERNEL_SP(tp)
150      ld ra, 0(sp)
151      ld s0, 16(sp)
    …
163      ld sp, SWITCH_TO_SP(sp)
164      addi sp, sp, SWITCH_TO_SIZE
165      ret
166  ENDPROC(switch_to)
```

注意切换发生的分界：把旧sp写入prev的PCB后，tp改指向next，sp改为next的栈。之后恢复的ra属于next，最后的ret自然回到next保留的返回位置。

为什么只保存这14个寄存器

这里的汇编入口是C调用约定中的普通函数边界，ra、sp和callee-saved的s0—s11足以维护这种内核调用链的恢复；caller-saved寄存器由正常调用约定处理。发生U态异常或异步中断时，原任务任意时刻的通用寄存器则先由完整异常帧保存，不能只靠这112字节。

为什么“返回”后像进入了另一个函数

A第一次切到B时，如果B曾经运行过，B恢复的是**自己上一次切出时的调用栈**，于是从自己的 `switch_to()` 调用后继续，完成自己的 `irq_restore(flags)`，再层层返回。A的局部变量和返回链仍留在A的栈里，等A未来被选中后才继续执行。新任务则走上一页的特殊首次返回路径。

纸上跟读： 画A和B两列，每列依次写 `sys_yield → do_scheduler → switch_to`。把两次“恢复另一个sp”的位置连起来，再标出哪一个任务最终执行哪一个ret。能画对这张图，就理解了上下文切换的核心。

8. Task2的锁：先睡下，再接收所有权

阅读入口：P2 kernel/locking/lock.c:30、P2 kernel/locking/lock.c:53、P2 kernel/locking/lock.c:67，再联读 P2 kernel/sched/sched.c:51 和 P2 kernel/sched/sched.c:62。

`sys_mutex_init(key)` 返回锁槽位下标。同一个key复用同一把锁，不是每次调用都分配新锁。每把锁同时保存status、owner及一个等待队列；它们分别回答“是否占用”“谁持有”“还有谁在等”。

获取锁先做一次原子try-acquire。成功则记录当前pid为owner；失败则调用 `do_block()`，把自己标记BLOCKED、放入该锁的等待队列，再调度其他任务。底层虽有自旋锁辅助接口，**这个mutex的竞争失败路径是阻塞，不是一直循环抢锁**。

跟一次会真正发生竞争的交错

1. A获取锁，owner=A，锁状态为LOCKED。
2. A在临界区内主动yield。A仍持锁，只是暂时不运行。
3. B运行并获取同一把锁失败。B进入锁等待队列，状态BLOCKED，不回ready_queue。
4. A恢复并释放锁。释放函数先把owner直接设为队首B，再调用 `do_unblock(B)`；锁仍保持LOCKED。
5. B以后被调度回来，沿自己原来的 `do_block()` 返回，检查owner确为自己，然后继续临界区。

释放代码的关键分支是：

```
72      if (!list_empty(&lock->block_queue)) {
73          list_node_t *node = lock->block_queue.next;
74          lock->owner = list_entry(node, pcb_t, list)->pid;
75          do_unblock(node); /* lock remains LOCKED until this owner releases */
76      } else {
77          lock->owner = 0;
78          spin_lock_release(&lock->lock);
79      }
```

这是一种**所有权直接移交**。有等待者时不先把锁改成UNLOCKED，所以新来的C不能在B真正获得CPU前把锁抢走。没有等待者时才清owner并真正解锁。

`do_unblock()` 的工作只是从等待队列摘除B，标READY，再放入就绪队尾；它没有立即执行B，也没有自行决定锁归谁。锁的owner由释放函数先设置好，两层职责要分开。

阅读 P2 test/validation/mutex1.c:8。测试特意在持锁期间yield，才能让另一任务在阶段2的非抢占环境下进入竞争路径。如果两个任务总是“拿锁 → 做完 → 放锁 → 最后才yield”，屏幕交替并不能证明阻塞队列实现正确。

本版本不支持递归获取；无效锁号、非owner释放等会触发断言。队列和锁状态修改由本地关中断包围，这是单核实现的前提，不能据此推导多核上也已经安全。

9. Task3的系统调用：沿一次sys_write走到底

阅读顺序：P2 tiny_libc/syscall.c:6 → P2 arch/riscv/kernel/entry.S:171 → P2 kernel/irq/irq.c:15 → P2 kernel/syscall/syscall.c:3 → P2 init/main.c:34。

```
用户printf在自己的栈里格式化字符串
 → sys_write → invoke_syscall：参数a0…a4，调用号a7
 → ecall：硬件转入stvec所指的S态入口
 → exception_handler_entry：SAVE_CONTEXT，传入异常帧指针
 → interrupt_helper：按scause分派到handle_syscall
 → syscall[number] → k_write → screen_write
 → 返回值写到异常帧的a0槽位
 → RESTORE_CONTEXT → sret → 用户ecall之后继续
```

这里的 `regs->regs[17]` 是保存的a7，`regs->regs[10]` 是保存的a0。`handle_syscall()` 的实现很短，但要能解释每一行：

```
 6      uint64_t number = regs->regs[17]; /* a7 */
 7      regs->sepc += 4; /* ecall is always a 32-bit instruction */
 8      if (number >= NUM_SYSCALLS || !syscall[number]) {
 9          regs->regs[10] = (reg_t)-38; /* ENOSYS */
10          return;
11      }
12      regs->regs[10] = syscall[number](regs->regs[10], regs->regs[11],
13                                       regs->regs[12], regs->regs[13], regs->regs[14]);
```

系统调用表在 P2 init/main.c:173 初始化。不是所有编号都有实现；超出表界或空表项都返回 `-38`。输出字符串的 `k_write()` 先验证完整字符串位于当前任务的镜像槽或用户栈中并找到终止符，不符合时返回 `-14`。这些检查不等于已经建立页表级内存隔离。

三张“表/入口”分别解决什么

内核跳转表位于固定地址，早期阶段用函数指针直接调用服务。**系统调用表**是内核里的 `syscall[]`，阶段3/4先经ecall和异常分派，才索引到它。**BIOS入口**是课程固件提供的固定接口，内核用它做磁盘、定时器、串口等底层操作。

同一个 `sys_write()` 在阶段1/2走 `call_jmptab()`，在阶段3/4才走 `invoke_syscall()`。阅读时打开对应的 `#if LAB_STAGE` 分支；不要因为函数名没变，就假定两个阶段的权限边界也相同。

如果一次系统调用内部阻塞，当前任务会连同尚未返回的内核调用栈一起暂停。恢复后继续完成处理，最终通过原异常帧返回用户态；并不会从用户程序main重新开始。

10. 异常入口：tp、sscratch和两种现场

聚焦 P2 arch/riscv/kernel/entry.S:8 的 `SAVE_CONTEXT`、P2 arch/riscv/kernel/entry.S:70 的 `RESTORE_CONTEXT`，以及 P2 arch/riscv/kernel/trap.S:3 的入口安装。

硬件把异常返回位置和原因等写入CSR并按stvec跳入S态，但不会替这份内核自动保存全部通用寄存器，也不会自动把用户sp换成本工程的内核栈。这些工作由entry.S完成。

先跟这五个动作，别一开始就背全部sd/ld

1. `csrrw tp,sscratch,tp`：原sscratch里的PCB进入tp；原用户tp暂存在sscratch中。
2. 把原t0先存到PCB的 `trap_t0`，再读取sstatus的SPP位，判断异常来自U态还是S态。
3. 来自U态则保存用户sp，从PCB取内核sp；来自S态则沿用当前S态栈。随后减288字节建立异常帧。
4. 保存其余寄存器和CSR；从sscratch取回原tp写入异常帧，再把sscratch恢复成PCB地址。
5. a0传异常帧指针，a1/a2传stval/scause，调用C函数 `interrupt_helper()`。

入口最早的几行是：

```
 8  .macro SAVE_CONTEXT
 9      csrrw tp, CSR_SSCRATCH, tp
10      sd t0, PCB_TRAP_T0(tp)
11      csrr t0, CSR_SSTATUS
12      andi t0, t0, SR_SPP
13      bnez t0, 1f
14      sd sp, PCB_USER_SP(tp)
15      ld sp, PCB_KERNEL_SP(tp)
16  1:  addi sp, sp, -OFFSET_SIZE
```

`RESTORE_CONTEXT` 恢复sstatus和sepc，更新PCB中可供下次U态陷入使用的内核栈位置，再恢复通用寄存器。最后才恢复原tp和原sp，因此恢复前仍能使用内核PCB及异常栈。紧随其后的 `sret` 按CSR内容返回相应特权级。

用三个问题区分容易混淆的现场

“用户刚才执行到哪？” 看异常帧中的sepc。**“内核调用链以后从哪继续？”** 看切换帧保存的ra和sp。**“这是哪个任务的现场？”** 看当前PCB与它指向的内核栈。这三者互相配合，不能只看一个入口地址。

ecall是同步的服务请求，返回时应越过已执行的32位ecall，所以 `sepc+=4`。时钟是异步打断，没有一条“请求指令”需要跳过，不能照抄这个加4。`interrupt_helper()` 通过scause的中断标志位区分两张分派表，定时器处理器不会修改sepc。

自测： 若用户把tp改成一个普通数值，异常入口为何仍能找到PCB？若只保存ra/sp，不保存a0—t6，时钟打断任意计算会出现什么问题？

11. sleep、时钟和idle：谁让等待者再次有资格运行

阅读入口：P2 kernel/sched/sched.c:39、P2 kernel/sched/time.c:34、P2 kernel/irq/irq.c:27、P2 init/main.c:208。

`do_sleep(seconds)` 在seconds为0时只让出CPU；否则计算绝对截止时刻 `get_ticks()+seconds×time_base`，记录到PCB，然后把当前任务放入sleep_queue并阻塞。时间单位先是硬件tick，`time_base` 来自固件的时钟频率，不能把tick数直接当秒。

`check_sleeping()` 扫描睡眠队列，把所有到期任务逐个 `do_unblock()`。它先保存 `next=node->next`，再可能删除当前节点，避免删除后破坏遍历。这里并没有按截止时刻排序，所以不能只检查队首，也不能唤醒一个后就退出。

阶段3没有周期时钟中断，时间仍在前进

`rdtime` 读取的硬件计数仍然增加。阶段3在调度时检查到期任务；所有普通任务都睡眠时，pid0持续调度/检查，因而仍能发现到期者。代价是idle轮询，而不是“没有时钟中断就无法读取时间”。

阶段4把唤醒和轮转接到定时器上

初始化先设置第一次timer并使能STIE。随后每次S态时钟中断都走：

```
异常入口保存当前任务
 → interrupt_helper → handle_irq_timer
 → 设置下一个timer → 刷新屏幕 → do_scheduler
 → 检查到期睡眠者 → 轮转选择任务 → 恢复被选中的现场
```

设置下一次timer放在处理器开头，避免只触发一次。`TIMER_INTERVAL=150000` 的单位是tick；时间间隔需要除以实际time_base，不能脱离平台直接写成固定多少毫秒。

pid0既是启动上下文，也是没有普通READY任务时的执行载体。阶段4中它使用WFI等待，再开启全局SIE处理待决中断；源码对WFI与单独中断使能的关系有说明。若把idle简单改成不会处理唤醒的死循环，所有任务一起sleep时系统就可能再无进展。

单核中断约定也属于算法的一部分

`irq_save()` 保存旧sstatus并清SIE；`irq_restore()` 只在原来开中断时重新打开。处理器进入内核期间不随意开SIE，从而避免在半更新的队列上再次抢占。这是本版本的单核约定；它没有同时解决多个CPU并发修改同一队列的问题。

跟读任务： 假设A、B都睡眠，A先到期。从pid0开始画出“检测到期 → READY → 被选择 → 从原sys_sleep调用后继续”的完整路径。

12. 把现象与代码对应：四阶段跟读实验

优先阅读的小测试：P2 test/validation/coop1.c:3、P2 test/validation/mutex1.c:4、P2 test/test_project2/sleep.c:6、P2 test/validation/probe1.c:19。

阶段1看主动切换。 `coop1/coop2` 每轮都调用sys_yield。先把“谁入队、谁出队”写出来，再核对两个计数都能增长；仅看到一条LOAD信息不代表任务真的执行。

阶段2看锁竞争。 mutex测试使用同一个key，并在持锁期间yield。跟踪owner和B所在的队列，直到B真正从原获取函数返回。不要仅凭两条打印交替就判断互斥正确。

阶段3看ecall与睡眠。 sleep任务暂停一段时间后继续，coop_timer在其他任务睡眠时仍能推进。把一次用户sys_sleep追到内核阻塞，再追到未来的异常返回，确认没有重新进入用户main。

阶段4看异步抢占。 probe中的 `register_check()` 在一段计算里不调用yield或系统调用。两边都在任意一边完成计算前开始执行，才提供了抢占发生的证据；随后还检查寄存器、锁移交和睡眠下界。

一次输出也能帮助你定位层次

P2 tiny_libc/printf.c:220 在用户栈的buffer中格式化，再调用sys_write和sys_reflush；内核 `k_write → screen_write → screen_write_ch` 更新共享屏幕缓冲，光标位置保存在各自PCB里。P2 drivers/screen.c:121 比较新旧屏幕，只将变化位置通过BIOS串口输出。因此原始日志含VT100光标序列，数字更新有时只产生单个字符，不能简单按换行推断执行次序。

在现有环境复现

课程环境中，在解压后的Project2-Acore目录使用 `make run LAB_STAGE=1`，依次将阶段改为2、3、4；阶段4可指定 `TASKS='probe1 probe2'`。这些命令依赖课程QEMU、U-Boot和对应交叉编译器。

当前云环境可在仓库根目录运行 `python3 scripts/acore.py test`。通用QEMU测试使用临时源码副本、地址重定位与BIOS适配层，课程源码保持原地址。本次已重新执行四个阶段的通用QEMU检查，四阶段均通过，结果见“运行记录”。prj1本次进行了源码恢复及静态核对，未声称完成独立课程启动验收。

课程QEMU、loadbootd/DASIC与Pynq的结果需要在对应环境实际确认。`test/host-qemu/` 是通用QEMU测试适配层，不属于课程正常引导路径。

13. 合上代码后的检查题与建议答案

1. tasks数组和pcb数组能合成一个概念吗？ 不能。前者是从镜像读出的静态名称/位置/大小/入口，后者还有状态、队列、栈和睡眠截止时刻等运行信息。

2. prj1任务返回后为什么又看见输入提示？ 加载器以函数指针调用入口，任务crt0保存并恢复返回地址，最后沿正常调用链回到交互循环。

3. 新任务从没被中断，为何能走异常返回？ `init_pcb_stack()` 预先布置了两种现场，switch帧的ra指向ret_from_exception，异常帧的sepc指向程序入口。

4. kernel_sp总是内核栈最高地址吗？ 不是。它在U态运行前可指向可用内核栈位置，任务在内核中被切出后会指向当前保存的切换现场。读它必须说明当前阶段。

5. A yield后，A的do_scheduler就立即返回了吗？ 没有。A可能已暂停，CPU正在B的栈里继续。A自己未来恢复后，才完成A的剩余调用链。

6. 只保存ra/sp/s0—s11是否足够处理任意时钟中断？ 不够。这是switch_to一层的函数边界现场；异常入口还必须保存任意时刻的完整寄存器状态及CSR。

7. 释放锁后，等待者是否立即执行？ 只是变READY，何时运行由调度决定。本版本先移交owner并保持LOCKED，保证它未来返回获取函数时已拥有锁。

8. 系统调用或中断一定会切换任务吗？ 不一定。普通输出可能处理后回到同一任务；yield、阻塞或时钟轮转会进入调度，但队列状态仍可能让prev与next相同。

9. 无周期时钟中断的阶段3如何醒来？ 硬件rdtime仍在推进，其他任务的调度路径以及pid0的轮询调用check_sleeping，让到期任务回到就绪队列。

10. 将时钟处理也写成sepc加4，会怎样？ 会越过原任务本来应该执行的指令。只有这里的ecall返回路径明确需要跳过已完成的请求指令。

11. 用户tp不是PCB，内核如何恢复身份？ sscratch保存当前PCB；入口先交换tp与sscratch，再把原用户tp保存到异常帧，结束时恢复它。

12. 有用户栈、U态和指针检查，就有完整进程隔离了吗？ 没有。还须看页表或其他硬件保护。本版本只是教学内核的有限内存约束，没有实现完整多进程资源体系和退出回收。

五条必须能自己画出的链

`镜像工具→bootblock→head→main→loader→crt0；init_pcb_stack→switch_to→ret_from_exception→sret；yield→排队→换sp→另一个ret；获取失败→BLOCKED→移交owner→READY→继续获取；时钟→重设timer→检查sleep→轮转→异常返回。`

如果某条链画不通，只回到相应小节的源码入口补读。完成后，再读屏幕格式化、完整CSR位定义和更长的测试程序，作为第二轮细化。

TODO 导读：把“待补代码”变成能解释的契约

以下按你需要自己完成的TODO组织。F2表示从课程框架差异恢复的**未填实现版本**；P1是保存的实验一；P2是已完成的A-Core。后续代码直接摘自这些版本，保留文件行号，没有把伪代码冒充工程源码。

每组都先看原TODO，再读实现，最后回答三个问题：输入是什么；这段必须改变哪些状态；完成后下一步由谁继续。出现“节选”时，仅省略文中明确标出的行，完整文件仍可在源码索引打开。

按实验任务找对应代码

- **p1-task1**：BIOS打印与bootblock入口，见A2。
- **p1-task2**：读入内核、跳转、清BSS、建立C环境，见A2—A3。
- **p1-task3**：任务加载、crt0、任务main及返回，见A3—A4。
- **p1-task4**：task_info结构、镜像元信息、启动传参、跨扇区读取和按名称加载，见A1、A2、A4。
- **p2-task1**：链表、PCB、初始栈、调度、switch_to、早期跳转表，见A5—A10、A14、A20。
- **p2-task2**：自旋锁、互斥锁、阻塞与唤醒，见A9、A11—A13。
- **p2-task3**：用户调用封装、异常现场、系统调用、睡眠，见A7、A14—A20。
- **p2-task4**：时钟中断、重设定时器、轮转和安全地开启中断，见A8、A19。

完整的 TODO任务清单 逐条记录标记、文件位置、附录编号和状态；源码索引搜索 `TODO` 或 `F2:文件名` 可以查看原始空函数。A5中的PCB结构本身主要是框架提供的背景，列出它是为了读懂后续TODO会操作的字段。

写每一处TODO前，先填这五句话

1. 调用者传来什么，当前处于哪个特权级，使用哪个栈？
2. 当前任务/队列/锁的状态在进入时满足什么条件？
3. 我需要改哪些寄存器、内存字段或链表指针？
4. 这段可能阻塞、切换或永不返回吗？返回时结果放在哪里？
5. 哪个具体测试会在我漏写一步时失败？

不要把TODO标记直接当成未完成清单。 P1里不少标记下已经有代码；P2中的部分标记被实现替换。屏幕退格的P3 TODO和预实验test.S的p0 TODO不属于本次A-Core Task1—4；README的TODO则是需要完成项目说明，不是内核函数。

A1. p1-task4：镜像中的任务表与启动元信息

> **原TODO**（F2 tools/createimage.c:20）：
> `TODO: [p1-task4] design your own task_info_t`

同文件79、125、216行还要求填写任务记录、处理padding并写回镜像信息。输入是各ELF的加载数据与当前镜像位置；输出是镜像尾部的任务表，以及bootblock尾部约定位置的数量、偏移和内核大小。

```
 25  typedef struct {
 26      char task_name[TASK_NAME_LEN];
 27      uint32_t offset;
 28      uint32_t size;
 29      uint64_t entrypoint;
 30  } task_info_t;
    …
123              if(fidx>=2){
124                  strncpy(taskinfo[taskidx].task_name,*files,TASK_NAME_LEN-1);
125                  taskinfo[taskidx].task_name[TASK_NAME_LEN-1]='\0';
126                  taskinfo[taskidx].offset=task_start;
127                  taskinfo[taskidx].size=phyaddr-task_start;
128                  taskinfo[taskidx].entrypoint=get_entrypoint(ehdr);
129              }
    …
144          if (strcmp(*files, "bootblock") == 0) {
145              write_padding(img, &phyaddr, SECTOR_SIZE);
146          }
147
148          fclose(fp);
149          files++;
150      }
151      write_img_info(nbytes_kernel, taskinfo, tasknum, img);
```

已完成P2保留相同磁盘协议，写回引导区的核心是：

```
136      uint16_t n = count, bytes = kernel_size;
137      uint8_t signature[] = {0x55, 0xaa};
138      write_at(img, TASK_NUM_LOC, &n, sizeof(n));
139      write_at(img, APP_INFO_LOC, &app_offset, sizeof(app_offset));
140      write_at(img, OS_SIZE_LOC, &bytes, sizeof(bytes));
141      write_at(img, 0x1fe, signature, sizeof(signature));
```

必须理解： `task_start`是镜像中的字节位置；`entrypoint`来自ELF入口。这里记录的是磁盘任务元数据，并没有创建PCB。Task4阶段只要求bootblock补齐512B，不再给每个任务分配固定扇区数量。P2补齐镜像尾扇区是额外的完整读取处理。

易错与检验： 把内核字节数当扇区数写入，会让bootblock的向上取整再算错；改变结构体字段而不同时修改两端，会让名称之后的偏移全部错位。P1第230行TODO属于旧注释块，实际 `write_img_info()` 从267行起；不要只改那段已停用代码。

A2. p1-task1/2/4：bootblock的四个TODO

> **原TODO**（F2 arch/riscv/boot/bootblock.S:22）：
> `TODO: [p1-task2] call BIOS to read kernel in SD card`

同一文件的打印、传参和跳转TODO分别位于19、25、28行。下面是P1对应的完成代码，包含原有TODO标记。

```
21     // TODO: [p1-task1] call BIOS to print string "It's bootblock!"
22     la a0,msg
23     li a1,0
24     li a2,0
25     li a3,0
26     li a4,0
27     li a5,0
28     li a6,0
29     li a7,BIOS_PUTSTR
30     li t0,bios_func_entry
31     jalr t0
32
33     // TODO: [p1-task2] call BIOS to read kernel in SD card
34     li a0,kernel
35     li t0,os_size_loc
36     lhu t1,0(t0)
37     addi a1,t1,511
38     srli a1,a1,9
39     li a2,1
40     li a7,BIOS_SDREAD
41     li t0,bios_func_entry
42     jalr t0
43     // TODO: [p1-task4] load task-related arguments and pass them to kernel
44     li t0,task_num_loc
45     lhu a0,0(t0)
46     li t0,app_info_offset_loc
47     lwu a1,0(t0)
48
49
50     // TODO: [p1-task2] jump to kernel to start UCAS-OS
51     li t0,kernel
52     jr t0
```

逐段看： 21—31行调用BIOS打印；34—42行把内核字节数换算为扇区数，从扇区1读到kernel地址；44—47行重新准备内核main的a0/a1；51—52行跳转。调用BIOS时的参数会被消耗，不能沿用读盘前的a0充当任务数。

不变量与检验： 引导区字段宽度、内核链接地址和读入地址必须一致。只有看到内核main之后的输出，才证明完成了“读入并跳转”；仅出现bootblock打印只能证明第一个TODO工作了。

A3. p1-task2/3：为什么要先准备C运行环境

> **原TODO**（F2 arch/riscv/kernel/head.S:17）：
> `TODO: [p1-task2] clear BSS for flat non-ELF images`

下面先列P1内核入口的清零和建栈，再列任务crt0的保存、清零、调用与返回。

```
12  csrw CSR_SIE, zero
13  csrw CSR_SIP, zero
14
15
16  la t0,__bss_start
17  la t1,__BSS_END__
18
19  clear_bss:
20      beq t0,t1,clear_done
21      sd zero,0(t0)
22      addi t0,t0,8
23      j clear_bss
24
25
26  clear_done:
27
28  li sp,KERNEL_STACK
29
30
31  call main
```

```
 6      /* TODO: [p1-task3] setup C runtime environment for the user program */
 7      addi sp,sp,-16
 8      sd ra, 0(sp)
 9      sd s0, 8(sp)
10      addi s0, sp, 16
11
12      la t0, __bss_start
13      la t1, __BSS_END__
14
15      clear_bss:
16          bge t0, t1, clear_bss_done
17          sw zero, 0(t0)
18          addi t0, t0, 4
19          blt t0, t1, clear_bss
20      clear_bss_done:
21
22      /* TODO: [p1-task3] enter main function */
23      call main
24
25      /* TODO: [p1-task3] finish task and return to the kernel, replace this in p3-task2! */
26      ld ra,0(sp)
27      ld s0,8(sp)
28      addi sp,sp,16
29      jr ra
```

进入/离开条件： 内核main需要合法sp和已清零的BSS，同时必须保留a0/a1。任务crt0在P1中要维护加载器的返回链，所以先保存ra/s0，任务main返回后再恢复。

不能机械迁移： P2任务不是由加载器普通call启动，已由PCB提供独立栈；P2的crt0在main返回后循环yield。保留P1的“返回加载器”实现，会跳向不存在的调用者。验证时分别观察BSS检查、任务运行及返回行为，三者不是同一项检查。

A4. p1-task3/4：按名查找、跨扇区读入、执行

> **原TODO**（F2 kernel/loader/loader.c:9）：
> `TODO: 1. [p1-task3] load task from image via task id, and return its entrypoint 2. [p1-task4] load task via task name, thus the arg should be 'char *taskname'`

`init_task_info()`先用下列字节读取函数填充tasks数组；加载某任务时再查名称并读入代码。P1的核心实现如下。

```
12  void load_img_bytes(uint32_t offset,uint32_t size,void *dst)
13  {
14      unsigned char *out = (unsigned char *)dst;
15      while (size > 0)
16      {
17          uint32_t sector = offset / SECTOR_SIZE;
18          uint32_t in_sector = offset % SECTOR_SIZE;
19          uint32_t copy_size = SECTOR_SIZE - in_sector;
20          if (copy_size > size)
21          {
22              copy_size = size;
23          }
24          sd_read((unsigned)sector_buf, 1, sector);
25
26          for (uint32_t i = 0; i < copy_size; ++i)
27          {
28              out[i] = sector_buf[in_sector + i];
29          }
30          offset += copy_size;
31          out += copy_size;
32          size -= copy_size;
33      }
34  }
35  uint64_t load_task_img(char *taskname)
36  {
37      /**
38       * TODO:
39       * 1. [p1-task3] load task from image via task id, and return its entrypoint
40       * 2. [p1-task4] load task via task name, thus the arg should be 'char *taskname'
41       */
42       for(int i=0;i<task_num;i++){
43          if(strcmp(taskname,tasks[i].task_name)==0){
44              load_img_bytes(tasks[i].offset,tasks[i].size,(void *)tasks[i].entrypoint);
45              return tasks[i].entrypoint;
46          }
47       }
48       return 0;
```

加载器取得入口后这样执行：

```
129          uint64_t entry=load_task_img(taskname);
130          if(entry==0){
131              bios_putstr("Task not found\n\r");
132              continue;
133          }
134          ((void (*)(void))entry)();
135          bios_putstr("\n\rTask finished, back to loader\n\r");
```

输入/输出： `offset/size/dst`都是按字节描述；磁盘读盘接口则按扇区工作，临时sector_buf用于衔接二者。每轮推进out、offset并减少size，这三个动作缺一不可。按名查找返回入口，找不到返回0。

检验： 手算offset=1000、size=600的24/512/64字节三轮复制；用不存在的名字确认不会跳到0。P2完成版在复制任务后还执行fence.i，保证后续取指能看到刚写入的代码。

A5. 写调度TODO前：先读懂PCB里的对象

源码位置：P2 include/os/sched.h:57。这组结构体是理解TODO的背景，不能把已有字段都算成需要重新编写的功能。

```
57  typedef enum {
58      TASK_BLOCKED,
59      TASK_RUNNING,
60      TASK_READY,
61      TASK_EXITED,
62  } task_status_t;
63
64  /* Process Control Block */
65  typedef struct pcb
66  {
67      /* register context */
68      // NOTE: this order must be preserved, which is defined in regs.h!!
69      reg_t kernel_sp;
70      reg_t user_sp;
71      reg_t trap_t0; /* temporary saved before trap stack selection */
72      ptr_t user_stack_base;
73
74      /* previous, next pointer */
75      list_node_t list;
76
77      /* process id */
78      pid_t pid;
79
80      /* BLOCK | READY | RUNNING */
81      task_status_t status;
82
83      /* cursor position */
84      int cursor_x;
85      int cursor_y;
86
87      /* time(seconds) to wake up sleeping PCB */
88      uint64_t wakeup_time; /* absolute rdtime ticks */
89
90  } pcb_t;
```

按使用者理解字段： 汇编用kernel_sp恢复内核调用链；异常入口用user_sp和trap_t0保存原用户现场；链表API操作list；调度器修改status；sleep修改wakeup_time；屏幕代码读取当前PCB里的cursor。

运行中、就绪、阻塞不是三份PCB。它们是同一对象在状态和队列中的不同位置。一个PCB只有一个list节点，不能同时把它挂进两个队列。

trap_t0和user_stack_base是完成版为异常入口和参数检查补充的辅助字段。扩展PCB后，供汇编使用的偏移也必须同步核对，不能只让C结构体单方面改变布局。

检验： 在纸上给A画一个PCB，跟踪一次yield、一次获取锁失败、一次唤醒。每一步都写明status及list所属队列。只改status或只改链表都会破坏下一次调度。

A6. p2-task1：链表API要维持什么

> **原TODO**（F2 include/os/list.h:53）：
> `TODO: [p2-task1] implement your own list API`

输入是内嵌在PCB中的链表节点或队列头，输出是指针连接关系的变化，不是复制PCB。完成版的核心操作如下。

```
43  typedef struct list_node
44  {
45      struct list_node *next, *prev;
46  } list_node_t;
47
48  typedef list_node_t list_head;
    …
53  static inline void list_init(list_head *head)
54  {
55      head->next = head->prev = head;
56  }
57  static inline int list_empty(const list_head *head)
58  {
59      return head->next == head;
60  }
61  static inline void list_add_tail(list_node_t *node, list_head *head)
62  {
63      node->prev = head->prev;
64      node->next = head;
65      head->prev->next = node;
66      head->prev = node;
67  }
68  static inline void list_del(list_node_t *node)
69  {
70      node->prev->next = node->next;
71      node->next->prev = node->prev;
72      list_init(node);
73  }
74  static inline list_node_t *list_pop_front(list_head *head)
75  {
76      if (list_empty(head)) return NULL;
77      list_node_t *node = head->next;
78      list_del(node);
79      return node;
80  }
81  #define list_entry(ptr, type, member) \
82      ((type *)((char *)(ptr) - __builtin_offsetof(type, member)))
```

不变量： 空队列头的next/prev都指向自己；追加时四条连接关系必须成对更新；删除后把节点重新设为自环，后续do_block才能判断该节点尚未挂队列。

`list_entry`从成员地址减去成员偏移，得到PCB基址。它不会查表、不会分配内存，也不能把任意指针变成PCB。`list_pop_front`先判断空队列，返回的节点已经脱链。

检验： 给head、A、B画双向箭头，依次执行追加A、追加B、弹出A、删除B，核对head最终恢复自环。

A7. p2-task1/3：构造一次尚未发生过的返回

> **原TODO**（F2 init/main.c:69）：
> `TODO: [p2-task1] set sp to simulate just returning from switch_to NOTE: you should prepare a stack, and push some values to simulate a callee-saved context.`

同文件60行还要求初始化任务寄存器。先看两个现场结构，再看如何在新任务栈中摆放它们。

```
38  typedef struct regs_context
39  {
40      /* Saved main processor registers.*/
41      reg_t regs[32];
42
43      /* Saved special registers. */
44      reg_t sstatus;
45      reg_t sepc;
46      reg_t sbadaddr;
47      reg_t scause;
48  } regs_context_t;
49
50  /* used to save register infomation in switch_to */
51  typedef struct switchto_context
52  {
53      /* Callee saved registers.*/
54      reg_t regs[14];
55  } switchto_context_t;
```

```
124  static void init_pcb_stack(ptr_t ktop, ptr_t utop, ptr_t entry, pcb_t *task)
125  {
126      regs_context_t *regs = (regs_context_t *)(ktop - sizeof(*regs));
127      switchto_context_t *sw = (switchto_context_t *)((ptr_t)regs - sizeof(*sw));
128      bzero(regs, sizeof(*regs));
129      bzero(sw, sizeof(*sw));
130      regs->regs[2] = utop;
131      regs->sepc = entry;
132  #if LAB_STAGE >= 3
133      /* SPP=0 returns to U mode; SPIE controls SIE after sret. */
134      regs->sstatus = SR_SPIE;
135  #else
136      /* Cooperative tasks share S mode and call the jump table directly. */
137      regs->regs[4] = (ptr_t)task;
138      regs->sstatus = SR_SPP;
139  #endif
140      sw->regs[0] = (ptr_t)ret_from_exception;
141      sw->regs[1] = (ptr_t)sw;
142      task->kernel_sp = (ptr_t)sw;
143      task->user_sp = utop;
144  }
```

关键语句： `regs[2]`是任务sp槽，sepc指向首次入口；SPP决定sret返回U还是S。`sw->regs[0]`是未来ret要取出的ra，设为ret_from_exception；`sw->regs[1]`与kernel_sp共同指向预制切换帧。

检验： 沿 `switch_to→ret_from_exception→sret→crt0` 解释每一步谁提供跳转目标。如果把ra直接设成用户main，就绕开了异常现场恢复和特权级返回；如果把sp写成栈底，首次保存寄存器就会越过有效区域。

A8. p2-task1/4：把任务准备为可调度对象

> **原TODO**（F2 init/main.c:80）：
> `TODO: [p2-task1] load needed tasks and init their corresponding PCB`

83行还要求设置current_running。初始化全部完成之前，不应让时钟在半初始化的队列上运行。

```
146  static void init_pcb(void)
147  {
148      list_init(&ready_queue);
149      list_init(&sleep_queue);
150      list_init(&pid0_pcb.list);
151      pid0_pcb.status = TASK_RUNNING;
152      current_running = &pid0_pcb;
153      __asm__ volatile ("csrw sscratch, %0" :: "r"(current_running) : "memory");
154      for (int i = 0; i < task_num; ++i) {
155          pcb_t *task = &pcb[i];
156          bzero(task, sizeof(*task));
157          task->pid = process_id++;
158          task->status = TASK_READY;
159          task->cursor_y = i * 2;
160          list_init(&task->list);
161          ptr_t ktop = allocKernelPage(1) + PAGE_SIZE;
162          task->user_stack_base = allocUserPage(1);
163          ptr_t utop = task->user_stack_base + PAGE_SIZE;
164          uint64_t entry = load_task_img(tasks[i].task_name);
165          assert(entry);
166          init_pcb_stack(ktop, utop, entry, task);
167          list_add_tail(&task->list, &ready_queue);
168          printk("[LOAD] pid=%d task=%s entry=0x%lx\n\r", task->pid,
169                 tasks[i].task_name, entry);
170      }
171  }
    …
188  int main(int count, uint32_t app_offset)
189  {
190      init_jmptab();
191      init_task_info(count, app_offset);
192      init_pcb();
193      time_base = bios_read_fdt(TIMEBASE);
194      assert(time_base > 0);
195      init_locks();
196      init_syscall();
197      init_exception();
198      init_screen();
199      printk("[INIT] A-Core stage %d, %d tasks, timebase=%lu\n\r",
200             LAB_STAGE, task_num, time_base);
201  #if LAB_STAGE >= 4
202      bios_set_timer(get_ticks() + TIMER_INTERVAL);
203      enable_preempt();
204  #endif
```

逐步解释： pid0先成为当前任务；每个普通任务分配两种栈、加载映像、构造初始现场，再进入ready_queue。`load_task_img()`只负责程序数据，`init_pcb_stack()`负责未来怎样开始执行，两者缺一不可。

检验： 逐个核对pid、entry、栈顶、status和队列。time_base必须先取得有效频率；锁、系统调用表、异常表和屏幕都准备好后，才设置定时器及使能抢占。这里使用的是P2实际实现顺序，不是按TODO在模板中的文字位置随意开中断。

A9. p2-task1/2：调度与阻塞/唤醒是一套协议

> **原TODO**（F2 kernel/sched/sched.c:32）：
> `TODO: [p2-task1] Modify the current_running pointer.`

同文件35、50、55行分别要求切换、阻塞和唤醒。do_scheduler必须与do_block的状态修改保持一致。

```
16  void do_scheduler(void)
17  {
18      uint64_t flags = irq_save();
19      check_sleeping();
20      /************************************************************/
21      /* Do not touch this comment. Reserved for future projects. */
22      /************************************************************/
23      pcb_t *prev = current_running;
24      if (prev->status == TASK_RUNNING && prev != &pid0_pcb) {
25          prev->status = TASK_READY;
26          list_add_tail(&prev->list, &ready_queue);
27      }
28      list_node_t *node = list_pop_front(&ready_queue);
29      pcb_t *next = node ? list_entry(node, pcb_t, list) : &pid0_pcb;
30      if (prev == &pid0_pcb && next != prev) prev->status = TASK_READY;
31      next->status = TASK_RUNNING;
32      if (prev != next) {
33          /* switch_to installs tp and sscratch atomically with IRQs disabled. */
34          switch_to(prev, next);
35      }
36      irq_restore(flags);
37  }
    …
51  void do_block(list_node_t *node, list_head *queue)
52  {
53      uint64_t flags = irq_save();
54      assert(node == &current_running->list && current_running != &pid0_pcb);
55      assert(list_empty(node));
56      current_running->status = TASK_BLOCKED;
57      list_add_tail(node, queue);
58      do_scheduler();
59      irq_restore(flags);
60  }
61
62  void do_unblock(list_node_t *node)
63  {
64      uint64_t flags = irq_save();
65      pcb_t *task = list_entry(node, pcb_t, list);
66      assert(task->status == TASK_BLOCKED);
67      list_del(node);
68      task->status = TASK_READY;
69      list_add_tail(node, &ready_queue);
70      irq_restore(flags);
71  }
```

不变量： 只有仍为RUNNING的普通任务才回就绪队尾。do_block先标BLOCKED并进入等待队列，所以后续调度不会把它错误地再排回READY。do_unblock先从旧队列删除，再改READY并排队。

检验： 当前A持锁、B获取失败后，B必须只在锁等待队列中；如果这里无条件把prev加回ready_queue，B就会在条件仍不成立时继续执行。队列为空时选择pid0，不能解引用NULL。

A10. p2-task1：switch_to为何能返回另一个任务

> **原TODO**（F2 arch/riscv/kernel/entry.S:55）：
> `TODO: [p2-task1] save all callee save registers on kernel stack, see the definition of struct switchto_context in sched.h`

58行是恢复下一任务的对应TODO。输入a0为prev、a1为next，输出不是普通计算值，而是CPU寄存器与执行栈改为next的现场。

```
130  ENTRY(switch_to)
131      addi sp, sp, -SWITCH_TO_SIZE
132      sd ra, 0(sp)
133      sd sp, 8(sp)
134      sd s0, 16(sp)
135      sd s1, 24(sp)
136      sd s2, 32(sp)
137      sd s3, 40(sp)
138      sd s4, 48(sp)
139      sd s5, 56(sp)
140      sd s6, 64(sp)
141      sd s7, 72(sp)
142      sd s8, 80(sp)
143      sd s9, 88(sp)
144      sd s10, 96(sp)
145      sd s11, 104(sp)
146      sd sp, PCB_KERNEL_SP(a0)
147      mv tp, a1
148      csrw CSR_SSCRATCH, tp
149      ld sp, PCB_KERNEL_SP(tp)
150      ld ra, 0(sp)
151      ld s0, 16(sp)
152      ld s1, 24(sp)
153      ld s2, 32(sp)
154      ld s3, 40(sp)
155      ld s4, 48(sp)
156      ld s5, 56(sp)
157      ld s6, 64(sp)
158      ld s7, 72(sp)
159      ld s8, 80(sp)
160      ld s9, 88(sp)
161      ld s10, 96(sp)
162      ld s11, 104(sp)
163      ld sp, SWITCH_TO_SP(sp)
164      addi sp, sp, SWITCH_TO_SIZE
165      ret
166  ENDPROC(switch_to)
167  ENTRY(ret_from_exception)
168      RESTORE_CONTEXT
169      sret
170  ENDPROC(ret_from_exception)
```

分界点： 146行把旧sp交给prev；147—149行取得next的身份和栈。后面的ld/ret都从next的栈读取。新任务取出预制ra，已运行过的任务取出自己上次暂停的ra。

检验： 用A、B交替yield检查所有s0—s11是否保留。漏掉某个s寄存器，计数短期正常也不代表切换正确；只改tp而不换sp，两个任务仍会踩在同一条调用栈上。

A11. p2-task2：自旋锁和锁数组初始化

> **原TODO**（F2 kernel/locking/lock.c:20）：
> `TODO: [p2-task2] try to acquire spin lock`

同文件10、15、26、31行要求初始化锁数组、自旋锁初值、自旋获取与释放。它们是mutex下面的基础操作。

```
 8  void spin_lock_init(spin_lock_t *lock) { lock->status = UNLOCKED; }
 9  int spin_lock_try_acquire(spin_lock_t *lock)
10  {
11      return atomic_swap(LOCKED, (ptr_t)&lock->status) == UNLOCKED;
12  }
13  void spin_lock_acquire(spin_lock_t *lock)
14  {
15      while (!spin_lock_try_acquire(lock)) __asm__ volatile ("nop");
16  }
17  void spin_lock_release(spin_lock_t *lock)
18  {
19      atomic_swap(UNLOCKED, (ptr_t)&lock->status);
20  }
21  void init_locks(void)
22  {
23      for (int i = 0; i < LOCK_NUM; ++i) {
24          spin_lock_init(&mlocks[i].lock);
25          list_init(&mlocks[i].block_queue);
26          mlocks[i].used = 0;
27          mlocks[i].owner = 0;
28      }
29  }
```

队列和锁操作还依赖如下本地中断保护：

```
78  static inline uint64_t irq_save(void)
79  {
80      uint64_t old;
81      __asm__ volatile ("csrrc %0, sstatus, %1" : "=r"(old) : "r"(2UL) : "memory");
82      return old;
83  }
84  static inline void irq_restore(uint64_t old)
85  {
86      if (old & 2UL)
87          __asm__ volatile ("csrs sstatus, %0" :: "r"(2UL) : "memory");
88  }
```

输入/输出： atomic_swap把新值写进去并返回旧值；旧值为UNLOCKED才代表成功。try只尝试一次，自旋获取则不断尝试。init_locks还必须初始化每把锁的block_queue，而非只清一个status。

检验与边界： 解锁把状态写回UNLOCKED；volatile不能替代原子读改写。irq_restore恢复的是进入前的使能情况，不能无条件开中断；这些约定服务于当前单核内核，不等价于多核同步已经完成。

A12. p2-task2：同key复用及获取失败后阻塞

> **原TODO**（F2 kernel/locking/lock.c:36）：
> `TODO: [p2-task2] initialize mutex lock`

42行继续要求获取mutex。初始化函数返回锁下标；获取函数在成功或未来被移交所有权后才继续返回。

```
30  int do_mutex_lock_init(int key)
31  {
32      uint64_t flags = irq_save();
33      int free_slot = -1;
34      for (int i = 0; i < LOCK_NUM; ++i) {
35          if (mlocks[i].used && mlocks[i].key == key) {
36              irq_restore(flags);
37              return i;
38          }
39          if (!mlocks[i].used && free_slot < 0) free_slot = i;
40      }
41      if (free_slot >= 0) {
42          mlocks[free_slot].used = 1;
43          mlocks[free_slot].key = key;
44      }
45      irq_restore(flags);
46      return free_slot;
47  }
48  static mutex_lock_t *get_lock(int id)
49  {
50      assert(id >= 0 && id < LOCK_NUM && mlocks[id].used);
51      return &mlocks[id];
52  }
53  void do_mutex_lock_acquire(int id)
54  {
55      uint64_t flags = irq_save();
56      mutex_lock_t *lock = get_lock(id);
57      assert(lock->owner != current_running->pid); /* non-recursive */
58      if (spin_lock_try_acquire(&lock->lock)) {
59          lock->owner = current_running->pid;
60      } else {
61          do_block(&current_running->list, &lock->block_queue);
62          /* release handed ownership to us; no stealing by a new arrival. */
63          assert(lock->owner == current_running->pid);
64      }
65      irq_restore(flags);
66  }
```

为什么不能一律分配新锁： A、B用相同key是为了竞争同一个对象，重复初始化必须返回已有下标。没有空槽时返回−1，调用者应检查。

为什么这里是if而不是抢锁while： 获取失败后do_block让出CPU，释放者之后会把owner直接交给当前等待者。该任务恢复时检查owner就是自己，因此不再用一轮自旋与后来者竞争。

检验： 持锁任务在临界区yield，让同key的另一任务确实走到do_block。若双方使用了不同槽位，两者都能打印“获取成功”，却完全没有实现共同互斥。

A13. p2-task2：释放时移交owner，唤醒不等于立刻运行

> **原TODO**（F2 kernel/locking/lock.c:47）：
> `TODO: [p2-task2] release mutex lock`

先对应锁对象的三个状态维度，再看释放函数：

```
45  typedef struct mutex_lock
46  {
47      spin_lock_t lock;
48      list_head block_queue;
49      int key;
50      int used;
51      pid_t owner;
52  } mutex_lock_t;
```

```
67  void do_mutex_lock_release(int id)
68  {
69      uint64_t flags = irq_save();
70      mutex_lock_t *lock = get_lock(id);
71      assert(lock->owner == current_running->pid);
72      if (!list_empty(&lock->block_queue)) {
73          list_node_t *node = lock->block_queue.next;
74          lock->owner = list_entry(node, pcb_t, list)->pid;
75          do_unblock(node); /* lock remains LOCKED until this owner releases */
76      } else {
77          lock->owner = 0;
78          spin_lock_release(&lock->lock);
79      }
80      irq_restore(flags);
81  }
```

下面的测试安排能逼出竞争路径：

```
 6      int id = sys_mutex_init(42);
 7      assert(id >= 0 && id == sys_mutex_init(42));
 8      for (unsigned n = 0;; ++n) {
 9          sys_mutex_acquire(id);
10          sys_move_cursor(0, 0);
11          printf("[MUTEX 1] acquired %u\n", n);
12          sys_yield(); /* force the peer to contend while we own the lock */
13          sys_move_cursor(0, 0);
14          printf("[MUTEX 1] release %u\n", n);
15          sys_mutex_release(id);
16          sys_yield();
```

关键顺序： 有等待者时先设owner，再do_unblock，底层status仍为LOCKED；没有等待者才清owner并真正解锁。否则新到达者可能抢在刚唤醒的B之前拿走锁。

检验： 手画A持锁、B阻塞、A释放、C到达、B恢复的五步，核对C不能偷走已移交给B的锁。释放者不是owner、重复获取等情况在本实现中触发断言；它不是支持递归的锁。

A14. p2-task1/3：用户封装与内核系统调用分派

> **原TODO**（F2 tiny_libc/syscall.c:11）：
> `TODO: [p2-task3] implement invoke_syscall via inline assembly`

原框架还为各sys_*接口留下两类TODO：早期阶段经call_jmptab，阶段3/4经invoke_syscall。实现如下。

```
 6  static long invoke_syscall(long number, long p0, long p1, long p2, long p3, long p4)
 7  {
 8      register long a0 asm("a0") = p0;
 9      register long a1 asm("a1") = p1;
10      register long a2 asm("a2") = p2;
11      register long a3 asm("a3") = p3;
12      register long a4 asm("a4") = p4;
13      register long a7 asm("a7") = number;
14      asm volatile ("ecall" : "+r"(a0) : "r"(a1), "r"(a2), "r"(a3),
15                    "r"(a4), "r"(a7) : "memory");
16      return a0;
17  }
18  void sys_yield(void)
19  {
20  #if LAB_STAGE < 3
21      call_jmptab(YIELD, 0, 0, 0, 0, 0);
22  #else
23      invoke_syscall(SYSCALL_YIELD, 0, 0, 0, 0, 0);
24  #endif
25  }
   …
34  void sys_write(char *buff)
35  {
36  #if LAB_STAGE < 3
37      call_jmptab(PRINT, (long)buff, 0, 0, 0, 0);
38  #else
39      invoke_syscall(SYSCALL_WRITE, (long)buff, 0, 0, 0, 0);
40  #endif
41  }
```

内核处理器对应 F2 kernel/syscall/syscall.c:7 的TODO：

```
 3  void handle_syscall(regs_context_t *regs, uint64_t stval, uint64_t cause)
 4  {
 5      (void)stval; (void)cause;
 6      uint64_t number = regs->regs[17]; /* a7 */
 7      regs->sepc += 4; /* ecall is always a 32-bit instruction */
 8      if (number >= NUM_SYSCALLS || !syscall[number]) {
 9          regs->regs[10] = (reg_t)-38; /* ENOSYS */
10          return;
11      }
12      regs->regs[10] = syscall[number](regs->regs[10], regs->regs[11],
13                                       regs->regs[12], regs->regs[13], regs->regs[14]);
14  }
```

输入/输出： 调用号放a7，参数放a0—a4，结果通过保存帧的a0再恢复到用户a0。`+r(a0)`说明它既是输入又是输出，memory约束阻止跨系统调用错误重排相关内存访问。

其余封装同样按接口传参：cursor传x/y，mutex传key或id，sleep传秒数；get_timebase/get_tick需返回结果。只写ecall却没放正确调用号，会落入错误服务；只调用内核函数指针，则没有完成U态受控入口。

A15. p2-task3：SAVE_CONTEXT完整宏（上半）

> **原TODO**（F2 arch/riscv/kernel/entry.S:6）：
> `TODO: [p2-task3] save all general purpose registers here!`

这是完整SAVE_CONTEXT宏的前半，下一页连续给出后半。硬件不会替这段代码保存全部GPR和选择本内核的栈。

```
 8  .macro SAVE_CONTEXT
 9      csrrw tp, CSR_SSCRATCH, tp
10      sd t0, PCB_TRAP_T0(tp)
11      csrr t0, CSR_SSTATUS
12      andi t0, t0, SR_SPP
13      bnez t0, 1f
14      sd sp, PCB_USER_SP(tp)
15      ld sp, PCB_KERNEL_SP(tp)
16  1:  addi sp, sp, -OFFSET_SIZE
17      sd zero, 0(sp)
18      sd ra, 8(sp)
19      sd gp, 24(sp)
20      sd t1, 48(sp)
21      sd t2, 56(sp)
22      sd s0, 64(sp)
23      sd s1, 72(sp)
24      sd a0, 80(sp)
25      sd a1, 88(sp)
26      sd a2, 96(sp)
27      sd a3, 104(sp)
28      sd a4, 112(sp)
29      sd a5, 120(sp)
30      sd a6, 128(sp)
31      sd a7, 136(sp)
32      sd s2, 144(sp)
33      sd s3, 152(sp)
34      sd s4, 160(sp)
35      sd s5, 168(sp)
36      sd s6, 176(sp)
37      sd s7, 184(sp)
38      sd s8, 192(sp)
39      sd s9, 200(sp)
40      sd s10, 208(sp)
41      sd s11, 216(sp)
42      sd t3, 224(sp)
43      sd t4, 232(sp)
44      sd t5, 240(sp)
45      sd t6, 248(sp)
```

先读前九行： sscratch中的PCB被换进tp；原t0先放到PCB临时槽。根据SPP判断来源：来自U态时保存用户sp并取得kernel_sp，来自S态时沿用S栈；再减288字节分配异常帧。

再读成组sd： 每个寄存器都有固定槽位，偏移必须与regs_context_t、regs.h一致。用户可能在任意一条指令被时钟打断，所以这层不能只保存callee-saved寄存器。

检验： 分别设想U态ecall和S态idle被中断，确认不会把两种来源的栈混用。原t0、用户tp、原sp还未在本页全部恢复到最终槽位，必须继续读下一页。

A16. p2-task3：SAVE_CONTEXT后半及C处理入口

> **原TODO**（F2 arch/riscv/kernel/entry.S:18）：
> `TODO: [p2-task3] save sstatus, sepc, stval and scause on kernel stack`

本页接上SAVE_CONTEXT，同时对应72、75、80行的“保存现场、安排返回、调用interrupt_helper”TODO。

```
 46      ld t0, PCB_TRAP_T0(tp)
 47      sd t0, OFFSET_REG_T0(sp)
 48      csrr t0, CSR_SSCRATCH
 49      sd t0, OFFSET_REG_TP(sp)
 50      csrw CSR_SSCRATCH, tp
 51      csrr t0, CSR_SSTATUS
 52      sd t0, OFFSET_REG_SSTATUS(sp)
 53      andi t0, t0, SR_SPP
 54      beqz t0, 2f
 55      addi t0, sp, OFFSET_SIZE
 56      j 3f
 57  2:  ld t0, PCB_USER_SP(tp)
 58  3:  sd t0, OFFSET_REG_SP(sp)
 59      csrr t0, CSR_SEPC
 60      sd t0, OFFSET_REG_SEPC(sp)
 61      csrr t0, CSR_STVAL
 62      sd t0, OFFSET_REG_SBADADDR(sp)
 63      csrr t0, CSR_SCAUSE
 64      sd t0, OFFSET_REG_SCAUSE(sp)
 65      /* No FPU instructions in kernel or user binaries. */
 66      li t0, SR_SUM | SR_FS
 67      csrc CSR_SSTATUS, t0
 68  .endm
    …
171  ENTRY(exception_handler_entry)
172      SAVE_CONTEXT
173      mv a0, sp
174      ld a1, OFFSET_REG_SBADADDR(sp)
175      ld a2, OFFSET_REG_SCAUSE(sp)
176      call interrupt_helper
177      j ret_from_exception
178  ENDPROC(exception_handler_entry)
```

关键还原： 46—47行把原t0放进异常帧；48—50行把原用户tp从sscratch转存到帧，并重新让sscratch指向PCB。SPP为S时，原sp是建立帧前的栈地址；来自U时，原sp从PCB的user_sp取出。

CSR与参数： sepc决定之后从哪继续，stval/scause用于异常分派与诊断。a0传帧指针，a1传stval，a2传scause。不要在保存前覆盖这些原用户寄存器。

框架注释提出把ret_from_exception安排成返回去向；当前实现使用call之后显式跳转到ret_from_exception，达到相同控制流目标。理解返回链的含义，比机械照搬一句“把地址写入ra”更重要。

检验： 即便用户tp不是PCB，也必须保留它并在返回时恢复；中途sscratch约定若未恢复，下次异常就可能取错PCB。

A17. p2-task3：恢复现场与sret

> **原TODO**（F2 arch/riscv/kernel/entry.S:23）：
> `TODO: Restore all general purpose registers and sepc, sstatus`

66行还要求在ret_from_exception中使用宏并返回sepc。下面完整列出恢复宏和返回入口。

```
 70  .macro RESTORE_CONTEXT
 71      ld t0, OFFSET_REG_SSTATUS(sp)
 72      csrw CSR_SSTATUS, t0
 73      ld t0, OFFSET_REG_SEPC(sp)
 74      csrw CSR_SEPC, t0
 75      addi t0, sp, OFFSET_SIZE
 76      sd t0, PCB_KERNEL_SP(tp)
 77      csrw CSR_SSCRATCH, tp
 78      ld ra, 8(sp)
 79      ld gp, 24(sp)
 80      ld t1, 48(sp)
 81      ld t2, 56(sp)
 82      ld s0, 64(sp)
 83      ld s1, 72(sp)
 84      ld a0, 80(sp)
 85      ld a1, 88(sp)
 86      ld a2, 96(sp)
 87      ld a3, 104(sp)
 88      ld a4, 112(sp)
 89      ld a5, 120(sp)
 90      ld a6, 128(sp)
 91      ld a7, 136(sp)
 92      ld s2, 144(sp)
 93      ld s3, 152(sp)
 94      ld s4, 160(sp)
 95      ld s5, 168(sp)
 96      ld s6, 176(sp)
 97      ld s7, 184(sp)
 98      ld s8, 192(sp)
 99      ld s9, 200(sp)
100      ld s10, 208(sp)
101      ld s11, 216(sp)
102      ld t3, 224(sp)
103      ld t4, 232(sp)
104      ld t5, 240(sp)
105      ld t6, 248(sp)
106      ld t0, OFFSET_REG_T0(sp)
107      ld tp, OFFSET_REG_TP(sp)
108      ld sp, OFFSET_REG_SP(sp)
109  .endm
    …
167  ENTRY(ret_from_exception)
168      RESTORE_CONTEXT
169      sret
170  ENDPROC(ret_from_exception)
```

顺序不能随意换： CSR先从当前异常帧恢复；利用仍有效的内核tp更新kernel_sp/sscratch，再恢复各GPR；原用户tp和sp最后恢复，避免过早失去PCB或当前帧的访问基址。

检验： 返回用户态靠sret解释sstatus/sepc，不是普通ret。ecall的sepc已在系统调用处理器加4；时钟路径不加4。恢复错误的a0会丢失返回值，恢复错误的sstatus会改变返回权限或中断使能。

A18. p2-task3：sleep等待的是绝对截止tick

> **原TODO**（F2 kernel/sched/sched.c:41）：
> `TODO: [p2-task3] sleep(seconds)`

同模块调度器26行及time.c第36行要求检查睡眠队列并唤醒到期者。三个函数联合完成该协议。

```
39  void do_sleep(uint32_t seconds)
40  {
41      uint64_t flags = irq_save();
42      if (!seconds) {
43          do_scheduler();
44      } else {
45          current_running->wakeup_time = get_ticks() + (uint64_t)seconds * time_base;
46          do_block(&current_running->list, &sleep_queue);
47      }
48      irq_restore(flags);
49  }
```

```
 8  uint64_t get_ticks()
 9  {
10      __asm__ __volatile__(
11          "rdtime %0"
12          : "=r"(time_elapsed));
13      return time_elapsed;
14  }
    …
34  void check_sleeping(void)
35  {
36      uint64_t now = get_ticks();
37      list_node_t *node = sleep_queue.next;
38      while (node != &sleep_queue) {
39          list_node_t *next = node->next;
40          pcb_t *task = list_entry(node, pcb_t, list);
41          if ((int64_t)(now - task->wakeup_time) >= 0) do_unblock(node);
42          node = next;
43      }
44  }
```

输入/输出： sleep的参数是秒，rdtime返回tick，所以要乘time_base。seconds=0退化为yield；正值写绝对截止时刻，再用do_block进入sleep_queue，不是在原地忙等。

遍历约束： 先保留next，再可能do_unblock当前节点；一次扫描处理所有到期者。无排序保证时不能只看队首。阶段3由调度及pid0轮询检查，阶段4由定时器引发的调度反复检查。

检验： 让两个任务同刻到期，确认都变READY；让所有用户任务一起sleep，确认pid0仍能推动检查。把秒直接加到tick上，会导致睡眠时间远小于请求值。

A19. p2-task3/4：异常分派、时钟和入口安装

> **原TODO**（F2 kernel/irq/irq.c:15）：
> `TODO: [p2-task3] & [p2-task4] interrupt handler.`

原框架还要求填好两张表、安装stvec、处理timer。完成版对应如下。

```
15  void interrupt_helper(regs_context_t *regs, uint64_t stval, uint64_t scause)
16  {
17      uint64_t code = scause & ~SCAUSE_IRQ_FLAG;
18      if (scause & SCAUSE_IRQ_FLAG) {
19          if (code < IRQC_COUNT) irq_table[code](regs, stval, scause);
20          else handle_other(regs, stval, scause);
21      } else {
22          if (code < EXCC_COUNT) exc_table[code](regs, stval, scause);
23          else handle_other(regs, stval, scause);
24      }
25  }
26
27  void handle_irq_timer(regs_context_t *regs, uint64_t stval, uint64_t scause)
28  {
29      (void)regs; (void)stval; (void)scause;
30      bios_set_timer(get_ticks() + TIMER_INTERVAL);
31      screen_reflush();
32      do_scheduler();
33  }
34
35  void init_exception()
36  {
37      for (int i = 0; i < EXCC_COUNT; ++i) exc_table[i] = handle_other;
38      for (int i = 0; i < IRQC_COUNT; ++i) irq_table[i] = handle_other;
39  #if LAB_STAGE >= 3
40      exc_table[EXCC_SYSCALL] = handle_syscall;
41  #endif
42  #if LAB_STAGE >= 4
43      irq_table[IRQC_S_TIMER] = handle_irq_timer;
44  #endif
45      setup_exception();
46  }
```

```
 3  ENTRY(setup_exception)
 4      la t0, exception_handler_entry
 5      csrw CSR_STVEC, t0
 6      csrw CSR_SSCRATCH, tp
 7      li t0, 2 /* permit user rdtime when M-mode also permits it */
 8      csrs CSR_SCOUNTEREN, t0
 9      /* Enable only after all tables, queues and the timer are ready. */
10      ret
11  ENDPROC(setup_exception)
```

三处关键： scause最高位先区分中断和同步异常；所有未实现表项有默认错误处理器；时钟处理先重设下一次timer，再刷新和调度，不能只触发第一次。

关于开中断的TODO： 框架trap.S第8行要求实现全局使能，但当前版本把生效时机与主初始化、初始异常帧、idle结合安排。不能在表和队列尚未就绪时仅为了“填上这行”就打开中断。

检验： 非法原因落到handle_other；未知系统调用由syscall处理器返回错误；无yield计算仍能被轮转，才说明时钟抢占实际发生。TIMER_INTERVAL是tick，不是固定毫秒数。

A20. 两类服务表：TODO里相同的“初始化表”不代表同一件事

> **原TODO**（F2 init/main.c:45）：
> `TODO: [p2-task1] (S-core) initialize system call table.`

同文件89行是完整系统调用表的TODO。当前A-Core用两个函数分别安排早期跳转表和阶段3/4系统调用表。

```
 92  static void init_jmptab(void)
 93  {
 94      long (**table)() = (long (**)())KERNEL_JMPTAB_BASE;
 95      table[CONSOLE_PUTSTR] = k_putstr;
 96      table[CONSOLE_PUTCHAR] = k_putchar;
 97      table[CONSOLE_GETCHAR] = k_getchar;
 98      table[SD_READ] = k_sdread;
 99      table[SD_WRITE] = k_sdwrite;
100      table[QEMU_LOGGING] = k_logging;
101      table[SET_TIMER] = k_settimer;
102      table[READ_FDT] = k_fdt;
103      table[MOVE_CURSOR] = k_cursor;
104      table[PRINT] = k_write;
105      table[YIELD] = k_yield;
106      table[MUTEX_INIT] = k_lock_init;
107      table[MUTEX_ACQ] = k_lock_acq;
108      table[MUTEX_RELEASE] = k_lock_rel;
109      table[REFLUSH] = k_refresh;
110  }
    …
173  static void init_syscall(void)
174  {
175      bzero(syscall, sizeof(long (*)()) * NUM_SYSCALLS);
176      syscall[SYSCALL_YIELD] = k_yield;
177      syscall[SYSCALL_WRITE] = k_write;
178      syscall[SYSCALL_CURSOR] = k_cursor;
179      syscall[SYSCALL_REFLUSH] = k_refresh;
180      syscall[SYSCALL_SLEEP] = k_sleep;
181      syscall[SYSCALL_LOCK_INIT] = k_lock_init;
182      syscall[SYSCALL_LOCK_ACQ] = k_lock_acq;
183      syscall[SYSCALL_LOCK_RELEASE] = k_lock_rel;
184      syscall[SYSCALL_GET_TIMEBASE] = k_timebase;
185      syscall[SYSCALL_GET_TICK] = k_ticks;
186  }
```

输入/输出： 第一张表存放固定内存位置上的函数指针，由call_jmptab间接调用；第二张表是内核syscall数组，只有ecall、异常保存和分派之后才访问。表项索引必须和两端头文件中的编号一致。

k_*包装把各服务统一成接收五个long参数、返回long的调用形状；写屏幕、yield等无有意义返回值的服务也显式返回0。指针检查在k_write中完成，表里存一个地址本身并不会校验用户参数。

检验： 对照sys_yield在两个阶段实际走的分支；再追一次sys_write的a7编号，核对syscall表对应k_write。两张表初始化互换后即使能编译，运行时也会调错服务。

框架README中的TODO需要你说明自己的构建、实现范围与验证；它属于项目文档任务。P3退格、p0汇编预实验不在本次A-Core任务范围内，仍在完整TODO清单中保留并标记。